Foliantica

Technical Documentation

API Reference · Architecture · Setup Guide · Appendix

Version	1.0
Date	June 20, 2026
Stack	FastAPI · SQLAlchemy 2 · PostgreSQL · Next.js 14 · Electron
Test coverage	1 036 tests · 80 %+ overall

Contents

1 Architecture Overview

Foliantica is a **local-first, all-in-one writing studio** for authors. It runs entirely on the author's machine with no mandatory cloud dependency. The architecture is a layered monolith: an Electron shell wraps a Next.js web frontend and a FastAPI Python backend, all communicating over localhost. Optional Docker sidecars extend functionality (grammar checking, export, AI mention scanning) but are not required for core writing features.

1.1 System Layers

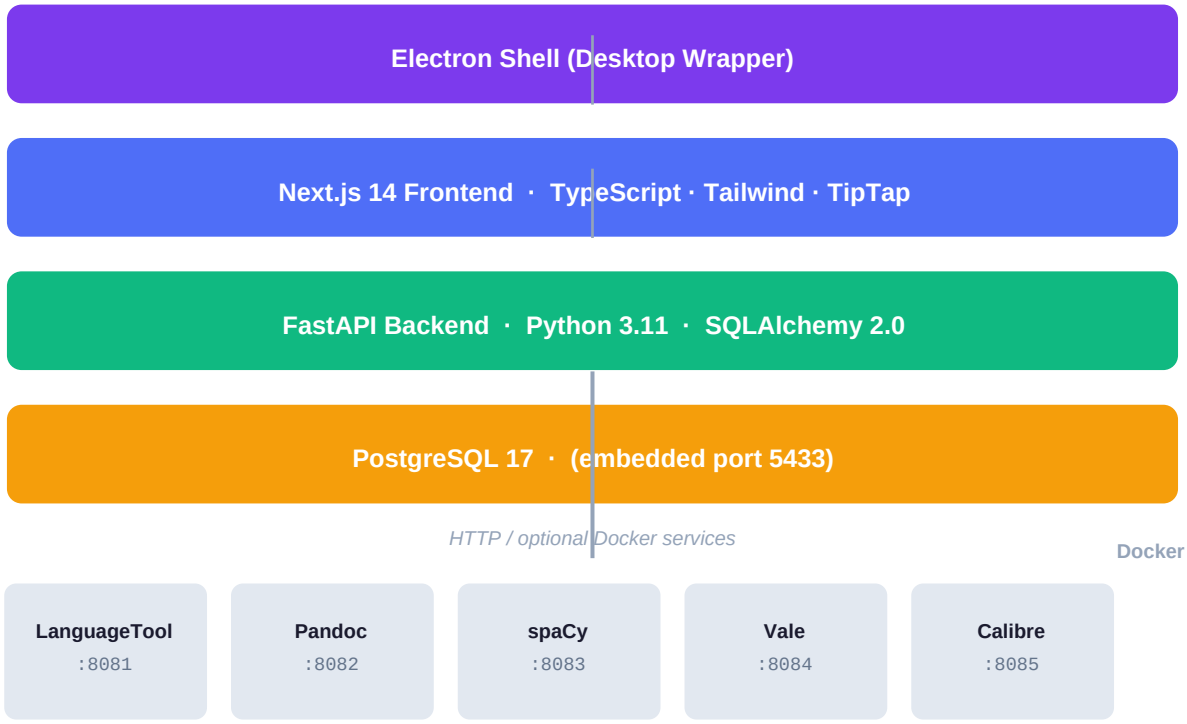

Figure 1 — Foliantica system layers. Arrows indicate HTTP communication. Docker sidecars are optional and communicate with the FastAPI backend only.

The backend exposes a REST API on `http://localhost:8799`. The Next.js frontend runs on port 3 000 and proxies all `/api/*` requests to the FastAPI backend. The Electron shell injects an `X-Foliantica-Host-Secret` header that grants full write access without a JWT token. Remote co-work guests connect over a Cloudflare Tunnel and authenticate with a short-lived JWT issued by the host.

1.2 Technology Stack

Component	Version	Purpose
FastAPI	0.109+	REST API framework, async, OpenAPI auto-docs
SQLAlchemy	2.0	ORM, declarative mapped_column, connection pool
PostgreSQL	17	Primary database (embedded on port 5433)

psycopg2	2.x	PostgreSQL driver
Alembic	1.x	Schema migrations
Cryptography (Fernet)	42+	API-key encryption at rest
Next.js	14	App Router, TypeScript, Tailwind CSS frontend
TipTap	3	Rich text editor with custom extensions
Zustand	5	Frontend UI state management
TanStack Query	5	Server state, caching, mutations
Electron	42	Desktop wrapper (Windows / macOS / Linux)
Pandoc (Docker)	3+	PDF / EPUB / DOCX export via LaTeX
LanguageTool (Docker)	6+	Grammar and style checking
Vale (Docker)	3+	Prose style rules
spaCy (Docker)	3+	Named entity recognition for mention scanning
pytest	8+	Python test suite (1 036 tests)
Playwright	1.x	End-to-end browser tests

1.3 Database Schema

The ORM is SQLAlchemy 2.0 with PostgreSQL 17 as the primary store (SQLite in WAL mode is supported as a fallback). All models use `mapped_column` with type annotations. JSON columns are used for flexible sub-structures (book metadata, time config, aliases, tags, inventory, export options).

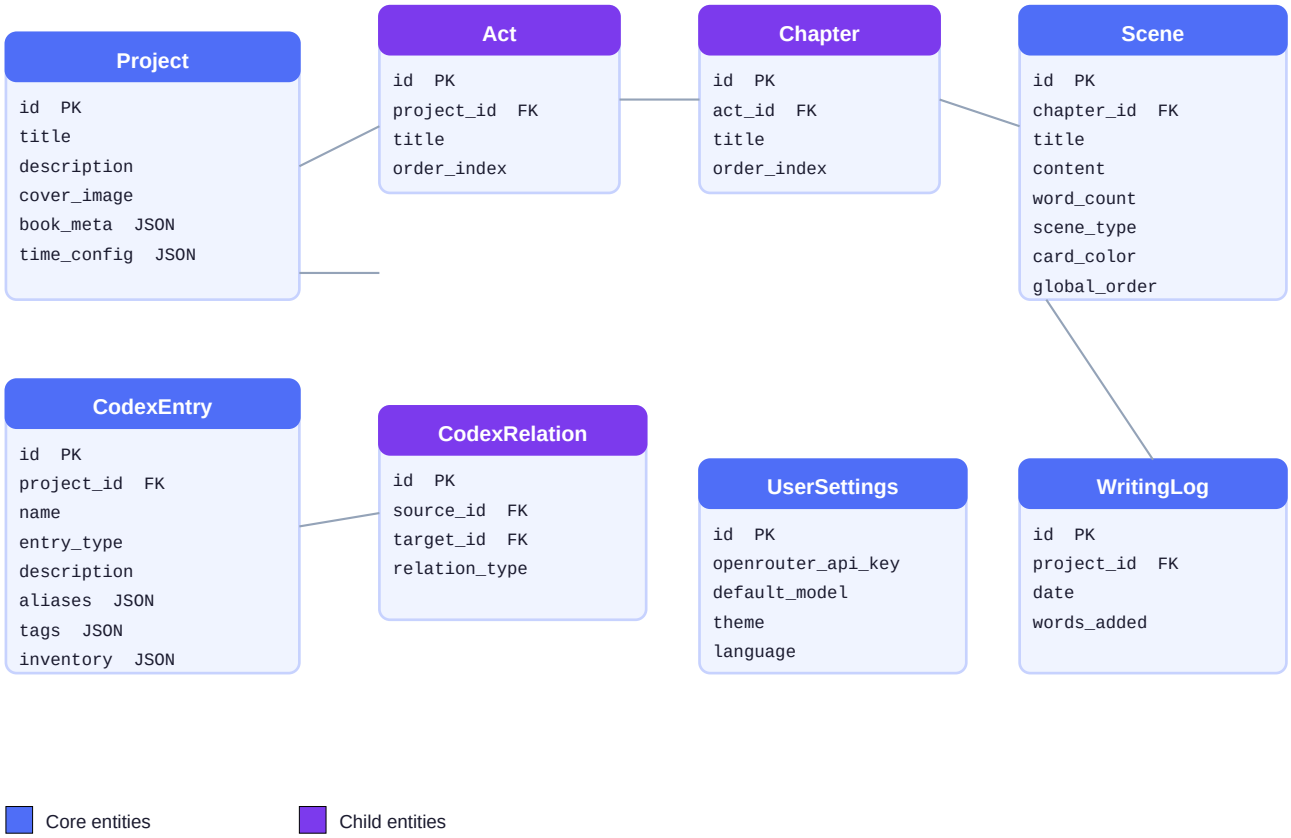

Figure 2 — Core ORM models (simplified). FK = Foreign Key, PK = Primary Key.

Model	Table	Key fields
`Project`	`projects`	title, description, book_meta (JSON), time_config (JSON)
`Act`	`acts`	project_id FK, title, order_index
`Chapter`	`chapters`	act_id FK, title, order_index
`Scene`	`scenes`	chapter_id FK, content, word_count, scene_type, card_color, global_order
`CodexEntry`	`codex_entries`	project_id FK, name, entry_type, aliases/tags/inventory (JSON)
`CodexRelation`	`codex_relations`	source_id FK, target_id FK, relation_type
`TimelineTrack`	`timeline_tracks`	project_id FK, name, color, track_type
`TimelineEvent`	`timeline_events`	track_id FK, title, scene_time (JSON)
`SceneVersion`	`scene_versions`	scene_id FK, content, sha256, label, kind
`WritingLog`	`writing_log`	project_id FK, date (YYYY-MM-DD), words_added
`Fragment`	`fragments`	project_id FK, tab, title, content, category
`ResearchItem`	`research_items`	project_id FK, url, text_content, tags (JSON)

`UserSettings`	`user_settings`	openrouter_api_key (encrypted), theme, language
`AIPrompt`	`ai_prompts`	name, system, user_template, is_built_in, built_in_key
`ExportProfile`	`export_profiles`	project_id (nullable), name, options_json
`PublisherProfile`	`publisher_profiles`	short_name, category, word_count_min/max
`QuerySubmission`	`query_submissions`	project_id FK, agent_name, status, date_sent
`SceneCommand`	`scene_commands`	scene_id FK, command_type, character_id, data (JSON)
`AchievementUnlock`	`achievement_unlocks`	key, unlocked_at, popup_shown_at
`ValeRuleEntry`	`vale_rule_entries`	lang, rule_name, entry_key, entry_value, enabled

2 Setup Guide

2.1 Prerequisites

Requirement	Version	Notes
Node.js	20 LTS+	For Next.js frontend build
Python	3.11+	Backend runtime
uv	0.4+	Fast Python package manager (replaces pip)
Docker Desktop	4+	For optional sidecars (LanguageTool, Pandoc, etc.)
Git	2+	Source checkout
PostgreSQL	17 (optional)	Via Docker or system install; embedded on port 5433

2.2 Running from Source

1. Clone the repository

```
git clone https://github.com/your-org/foliantica.git
cd foliantica
```

2. Install Python dependencies (api/)

```
cd api
uv sync
# activates .venv automatically
```

3. Install Node dependencies (web/)

```
cd ../web
npm install
```

4. Start optional Docker services

```
.\scripts\test-stack-start.ps1 # Windows
./scripts/test-stack-start.sh # macOS / Linux
```

5. Launch all services (dev mode)

```
.\LaunchFoliantica.bat # Windows
./launch.sh # macOS / Linux
# Opens http://localhost:3000
```

6. Run tests (Python)

```
cd api
.venv\Scripts\python.exe -m pytest --tb=short -q
```

2.3 Environment Variables

All variables have sensible defaults for local development. Only FOLIANTICA_SECRET_KEY is required in production (auto-generated on first run if absent).

Variable	Default	Description
`LW_PG_HOST`	`127.0.0.1`	PostgreSQL host
`LW_PG_PORT`	`5433`	PostgreSQL port (embedded cluster)
`LW_PG_USER`	`foliantica`	Database user
`LW_PG_PASS`	`foliantica`	Database password
`LW_PG_DB`	`foliantica`	Database name
`FOLIANTICA_SECRET_KEY`	`(auto-generated)`	Fernet key for API-key encryption
`FOLIANTICA_HOST_SECRET`	`(Electron only)`	Electron host authentication secret
`LT_LANGS`	`de,en`	LanguageTool language models (.env)
`PYTEST_CURRENT_TEST`	`(test runner)`	Disables seed data during test runs
`LW_CONFIG_FILE`	`(internal)`	Path to local config JSON (testable)

2.4 Docker Services

All Docker services are optional. Enable them via `docker compose --profile <name> up -d` or through the Settings → External Services UI.

Service	Port	Profile	Purpose
`PostgreSQL`	`5433`	postgres	Primary database (optional — embedded by default)
`LanguageTool`	`8081`	languagetool	Grammar check (needs Java heap 512 m–2 g)
`Pandoc`	`8082`	pandoc	PDF/EPUB/DOCX export (custom image with LaTeX)
`spaCy`	`8083`	spacy	Token-aware NER for mention scanning
`Calibre`	`8084`	calibre	eBook conversion (optional)
`Vale`	`8085`	vale	Style checking (optional)

3 API Reference

The FastAPI backend exposes **25 routers** with a combined total of **~140 endpoints**. All routes are prefixed with `/api`. The OpenAPI interactive docs are available at `http://localhost:8799/docs` when the backend is running.

[GET] get [POST] post [PATCH] patch [DELETE] delete [PUT] put

3.1 projects

Project management — CRUD, series, corkboard, structure

GET	/api/projects	List all projects
POST	/api/projects	Create new project
GET	/api/projects/{id}	Get project details
PATCH	/api/projects/{id}	Update project metadata
DELETE	/api/projects/{id}	Delete project (cascade)
GET	/api/projects/{id}/structure	Full act→chapter→scene tree
GET	/api/projects/{id}/corkboard	Scenes with layout positions
PATCH	/api/projects/{id}/corkboard-prefs	Update corkboard preferences
POST	/api/projects/{id}/connections	Create scene connection
DELETE	/api/projects/{id}/connections/{cid}	Delete scene connection
POST	/api/projects/{id}/corkboard/global-order	Reorder scenes chronologically
PATCH	/api/projects/{id}/subplot-names	Update subplot labels
GET	/api/projects/series	List series groupings
GET	/api/projects/{id}/pov-stats	POV character word distribution

3.2 acts

Top-level story divisions

GET	/api/projects/{id}/acts	List acts in project
POST	/api/acts	Create act
PATCH	/api/acts/{id}	Update act
DELETE	/api/acts/{id}	Delete act
POST	/api/acts/reorder	Batch reorder acts
GET	/api/acts/{id}/read	Read-mode flowing prose view

3.3 chapters

Mid-level groupings within acts

GET	/api/acts/{id}/chapters	List chapters in act

POST	/api/chapters	Create chapter
PATCH	/api/chapters/{id}	Update chapter
DELETE	/api/chapters/{id}	Delete chapter
POST	/api/chapters/reorder	Batch reorder chapters
GET	/api/chapters/{id}/read	Read-mode view

3.4 scenes

Smallest editable unit — content, versions, mentions

GET	/api/chapters/{id}/scenes	List scenes in chapter
POST	/api/scenes	Create scene
GET	/api/scenes/{id}	Get scene
PATCH	/api/scenes/{id}	Update scene content / metadata
DELETE	/api/scenes/{id}	Delete scene
POST	/api/scenes/reorder	Batch reorder
POST	/api/scenes/{id}/versions	Create version snapshot
GET	/api/scenes/{id}/versions	List version history
GET	/api/scenes/{id}/versions/{vid}	Get version content
POST	/api/scenes/{id}/versions/{vid}/restore	Restore to version
GET	/api/scenes/{id}/mention-stats	Codex mention counts
POST	/api/scenes/{id}/mentions/rescan	Re-scan mentions
GET	/api/projects/{id}/mention-stats	Project-wide mention stats
POST	/api/projects/{id}/mentions/rescan	Rescan all scenes
GET	/api/projects/{id}/writing-log	Daily word counts
GET	/api/writing-log	All projects writing log
GET	/api/projects/{id}/ghost-texts	AI inline suggestions

3.5 codex

World-building entries — characters, locations, items, lore

GET	/api/projects/{id}/codex	List entries with share filtering
POST	/api/codex	Create entry
GET	/api/codex/{id}	Get entry
PATCH	/api/codex/{id}	Update entry
DELETE	/api/codex/{id}	Delete entry
GET	/api/codex/{id}/access	Get explicit access list

PUT	/api/codex/{id}/access	Set explicit access list
GET	/api/codex/{id}/relations	Get all relations
POST	/api/codex/relations	Create relation
DELETE	/api/codex/relations/{rid}	Delete relation
GET	/api/codex/{id}/scene-mentions	Scenes mentioning this entry

3.6 time

Custom calendar, timeline tracks and events

GET	/api/projects/{id}/time-config	Get time system config
PATCH	/api/projects/{id}/time-config	Update time system
GET	/api/projects/{id}/timeline	Timeline grid (scenes × time)
GET	/api/projects/{id}/timeline-v2	Alternative timeline view
GET	/api/projects/{id}/timeline-tracks	List parallel tracks
POST	/api/projects/{id}/timeline-tracks	Create track
PATCH	/api/projects/{id}/timeline-tracks/{tid}	Update track
DELETE	/api/projects/{id}/timeline-tracks/{tid}	Delete track
GET	/api/projects/{id}/timeline-events	List events
POST	/api/projects/{id}/timeline-events	Create event
PATCH	/api/projects/{id}/timeline-events/{eid}	Update event
DELETE	/api/projects/{id}/timeline-events/{eid}	Delete event

3.7 ai

AI text generation, chat, synopsis, translation

POST	/api/ai/generate	Continue / rewrite / brainstorm
POST	/api/ai/ki	Inline KI node generation
POST	/api/ai/translate	Translate scene text
POST	/api/ai/structure	Analyse prose structure
POST	/api/ai/scenes/{id}/synopsis	Auto-generate synopsis
POST	/api/ai/chat	Context-aware chat sidebar

3.8 export

Manuscript export via Pandoc — PDF/EPUB/DOCX/Markdown/LaTeX

POST	/api/export/{id}/export	Export single format
GET	/api/export/fonts	Available font list
GET	/api/export/{id}/export-profiles	List export profiles

POST	/api/export/{id}/export-profiles	Create profile
PATCH	/api/export/export-profiles/{pid}	Update profile
DELETE	/api/export/export-profiles/{pid}	Delete profile
GET	/api/export/{id}/export/structure	Scenes to include
POST	/api/export/{id}/export/batch	Publisher pack (multi-format)

3.9 research

URL clips, text notes, images and PDFs

GET	/api/projects/{id}/research	List research items
POST	/api/projects/{id}/research	Create item
PATCH	/api/research/{id}	Update item (model_fields_set)
DELETE	/api/research/{id}	Delete item
POST	/api/research/{id}/fetch-url	Fetch / refresh URL metadata

3.10 analytics

Scene/chapter stats, Flesch scores, writing heatmap

GET	/api/analytics/projects/{id}/analytics	Full project analytics
GET	/api/analytics/stats/totals	Global totals + day-of-week
POST	/api/analytics/stats/ping	Record stats view

3.11 settings

App settings, AI providers, prompts, Docker, data folder

GET	/api/settings	Get all settings
POST	/api/settings	Update settings
GET	/api/settings/providers	List AI providers
POST	/api/settings/providers/active	Set active provider
POST	/api/settings/providers/{pid}	Set provider credentials
GET	/api/settings/providers/{pid}/models	List provider models
POST	/api/settings/providers/model-map	Map model → provider
GET	/api/settings/prompts	List AI prompts
POST	/api/settings/prompts	Create custom prompt
PUT	/api/settings/prompts/{pid}	Update prompt
DELETE	/api/settings/prompts/{pid}	Delete prompt
POST	/api/settings/prompts/{pid}/revert	Revert to factory default
GET	/api/settings/pg-config	Get PostgreSQL config

POST	/api/settings/pg-config	Save PostgreSQL config
GET	/api/settings/service-status	Docker service health
GET	/api/settings/data-dir	Current data folder
POST	/api/settings/data-dir	Change data folder

3.12 sync

Cloud backup & restore — Dropbox / Drive / OneDrive

GET	/api/sync/status	Sync status (enabled, mode, last sync)
POST	/api/sync/trigger	Manually trigger sync
POST	/api/sync/dump	Export DB to SQL backup file
POST	/api/sync/restore	Restore from SQL backup file

3.13 collab

Real-time co-work — invitations, JWT sessions, Cloudflare tunnel

GET	/api/collab/info	Co-work status
GET	/api/collab/invitations	List invitations
POST	/api/collab/invitations	Create invitation
PATCH	/api/collab/invitations/{id}	Update invitation
DELETE	/api/collab/invitations/{id}	Revoke invitation
GET	/api/collab/invitations/{id}/token	Get JWT token
POST	/api/collab/join	Guest join (exchange token)
GET	/api/collab/sessions	Active guest sessions
POST	/api/collab/kick/{sid}	Remove guest
GET	/api/collab/cloudflare/status	Cloudflare tunnel status
POST	/api/collab/cloudflare/open	Open tunnel
POST	/api/collab/cloudflare/close	Close tunnel

3.14 grammar

LanguageTool grammar and style checking

POST	/api/grammar/check	Check text (returns matches)
GET	/api/grammar/languages	Supported language list

3.15 prose

Vale style checker integration

POST	/api/prose/check	Vale style check on scene text

3.16 vale

Vale rule management — sync, edit, custom rules

GET	/api/vale/sync-status	Rule sync status

POST	/api/vale/sync-rules	Download / sync rules
GET	/api/vale/rule-meta/{lang}	Rule metadata
GET	/api/vale/rule-entries/{lang}/{name}	Rule entries
PATCH	/api/vale/rule-entries/{lang}/{name}	Toggle entry enabled
PUT	/api/vale/rule-entries/{lang}/{name}	Add custom entry
GET	/api/vale/custom-rules	List custom rules
PUT	/api/vale/custom-rules	Update custom rules

3.17 fragments

Snippets, ideas, archive, custom tabs

GET	/api/projects/{id}/fragment-tabs	Get tab list
PATCH	/api/projects/{id}/fragment-tabs	Update tabs
GET	/api/projects/{id}/fragments	List fragments
POST	/api/projects/{id}/fragments	Create fragment
PATCH	/api/fragments/{id}	Update fragment
DELETE	/api/fragments/{id}	Delete fragment

3.18 images

Upload and delete covers, codex images, research media

POST	/api/projects/{id}/cover	Upload project cover
DELETE	/api/projects/{id}/cover	Delete cover
POST	/api/codex/{id}/image	Upload codex entry image
DELETE	/api/codex/{id}/image	Delete entry image
POST	/api/research/{id}/media	Upload research media
DELETE	/api/research/media/{id}	Delete research media

3.19 scene_commands

Inventory / currency system — commands, balances, logs

POST	/api/scene_commands/projects/{id}/commands/resync	Resync state
POST	/api/scene_commands/scenes/{id}/commands/sync	Sync commands
GET	/api/scene_commands/projects/{id}/command-history	Project history

GET	/api/scene_commands/scenes/{id}/item-log	Items in scene
GET	/api/scene_commands/scenes/{id}/currency-balance	Currency balance
GET	/api/scene_commands/codex/{cid}/currencies	Character currencies
GET	/api/scene_commands/projects/{id}/item-log	Project items
GET	/api/scene_commands/projects/{id}/currency-log	Project currencies
GET	/api/scene_commands/codex/{cid}/item-log	Character items
GET	/api/scene_commands/codex/{cid}/currency-log	Character currencies
GET	/api/scene_commands/codex/{cid}/inventory-summary	Inventory summary

3.20 submissions

Agent / publisher query tracker

GET	/api/submissions/projects/{id}/submissions	List submissions
POST	/api/submissions/projects/{id}/submissions	Log new query
PATCH	/api/submissions/submissions/{id}	Update status
DELETE	/api/submissions/submissions/{id}	Delete entry

3.21 comments

Scene annotations for co-work guests

GET	/api/comments/scenes/{id}/comments	List annotations
POST	/api/comments/scenes/{id}/comments	Create annotation
PATCH	/api/comments/comments/{id}	Update annotation
DELETE	/api/comments/comments/{id}	Delete annotation
POST	/api/comments/scenes/{id}/comments/sync-positions	Sync positions

3.22 achievements

Achievement unlocks

GET	/api/achievements	List unlocked achievements
POST	/api/achievements/ack	Acknowledge / dismiss popup

3.23 graph

Relations graph — SVG radial mindmap

GET	/api/graph/{id}/graph	Codex relations graph (SVG)

3.24 imports

Story and codex import

POST	/api/imports/{id}/import/story	Import story (Markdown / JSON)
POST	/api/imports/{id}/import/codex	Import codex (CSV / JSON / MD)

4 Test Coverage

The Python test suite uses **pytest** with `pytest-cov`. All 1 036 tests run against a real PostgreSQL database (port 5433) — no mocks for the database layer. Coverage is measured against `api/routers/` only.

Metric	Value
Total tests	1 036
Failures	0
Errors	0
Overall router coverage	~80 %
Highest coverage	research.py (96 %)
Lowest coverage	ai.py (32 %) — streaming responses hard to mock

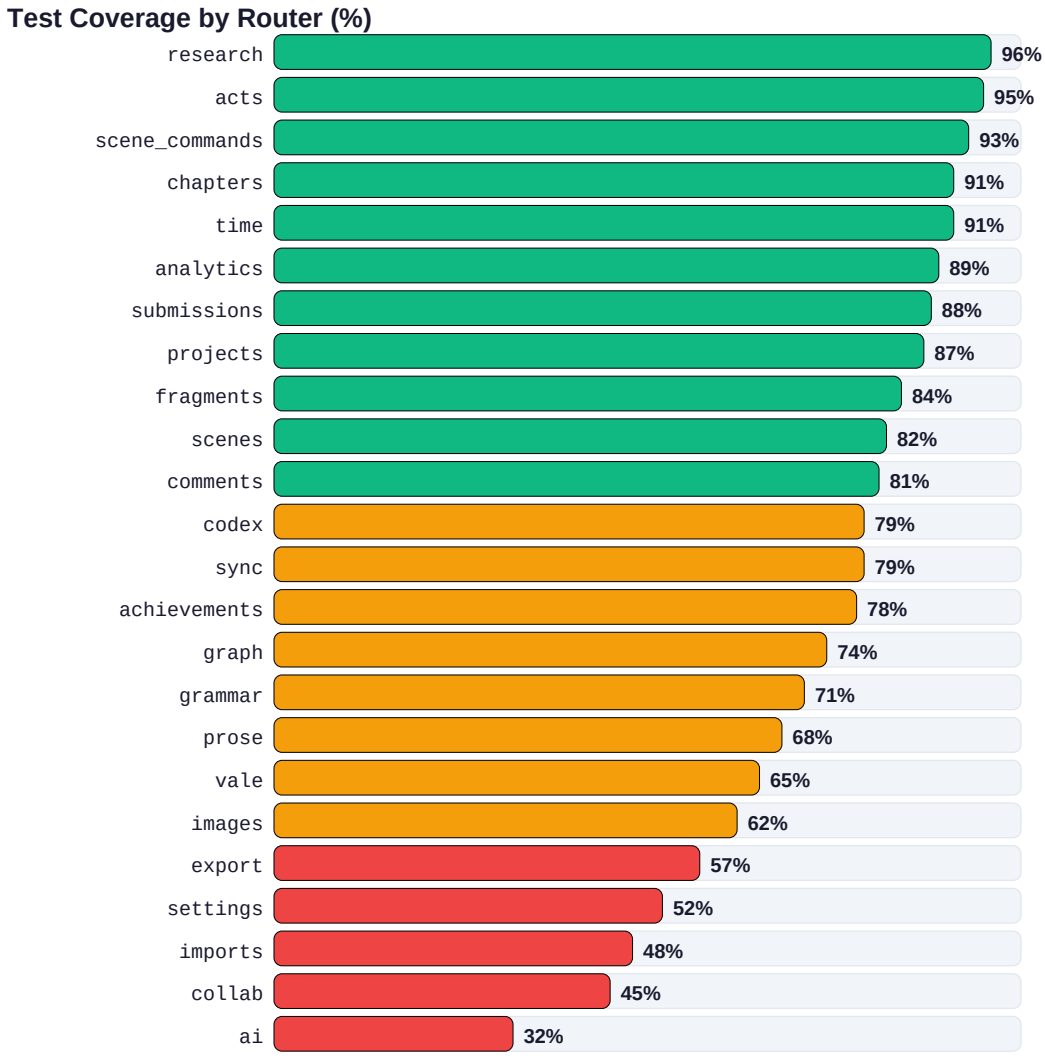

Figure 3 — Per-router test coverage. Green ≥ 80 %, amber 60–79 %, red < 60 %.

Low-coverage areas and reasons:

Router	Coverage	Reason
`ai.py`	32 %	Server-Sent Events streaming; async generator hard to drive in TestClient
`export.py`	57 %	Requires Pandoc Docker sidecar; skipped in offline CI
`imports.py`	48 %	Large file uploads; happy-path only tested
`collab.py`	45 %	WebSocket + Cloudflare Tunnel; needs live network
`settings.py`	52 %	Docker compose calls require process isolation

Appendix A — Key Function Descriptions

This appendix describes the most important functions in the Foliantica backend. Pure helper functions and trivial CRUD wrappers are omitted. File paths are relative to the `api/` directory.

main.py — lifespan()

Async startup/shutdown context manager. Waits up to 60 s for PostgreSQL (Docker may still be starting), creates all ORM tables via Base.metadata.create_all(), seeds AI prompts and publisher profiles, initialises the uploads directory, registers SQLAlchemy flush/commit hooks for co-work broadcasts, and initialises the sync subsystem. On shutdown closes the sync backup and any open Cloudflare Tunnel.

main.py — CoworkAuthMiddleware

Starlette middleware that enforces host/guest access control on every request. Public paths (/api/health, /api/collab/join, /api/collab/info) bypass auth. Loopback clients (127.0.0.1, ::1) are always trusted. Electron injects an X-Foliantica-Host-Secret header that grants full host access. Remote guests must supply a valid Bearer JWT. Write operations are blocked when the access_mode is read_only or appearance_only.

crypto.py — encrypt() / decrypt()

Fernet symmetric encryption for API keys stored in UserSettings. The key is loaded from the FOLIANTICA_SECRET_KEY env var, then ~/.foliantica/.secret_key, then migrated from a legacy CWD location. decrypt() returns None on failure (handles cross-machine key mismatch gracefully rather than raising).

database.py — get_db()

FastAPI dependency that yields a SQLAlchemy Session. Used in every router via Depends(get_db). The engine connects to PostgreSQL on 127.0.0.1:5433 with connection pool size 5, max_overflow 10.

routers/scenes.py — patch_scene()

Handles PATCH /api/scenes/{id}. Updates content, syncs word_count, records a WritingLog entry for the current date, broadcasts changes to connected co-work guests via the WebSocket hub, and invalidates cached mention stats when content changes.

routers/scenes.py — restore_version()

POST /api/scenes/{id}/versions/{vid}/restore. Creates a pre-restore snapshot (tagged 'pre-restore'), then overwrites scene content with the chosen version. SHA-256 deduplication prevents storing identical snapshots. Versions are pruned to 30 per scene and 30-day retention.

routers/codex.py — list_entries()

GET /api/projects/{id}/codex. Applies three-way share filtering: entries owned by the project are always included; entries with share_mode='all' from other projects are included; entries with share_mode='specific' are included only if a CodexEntryAccess row exists for this project.

routers/time.py — get_timeline()

GET /api/projects/{id}/timeline. Iterates all scenes in scene order, parses scene_time JSON (using the module-aliased _json.loads — important: module uses `import json as _json`), and builds a grid mapping scenes to time column positions. Exceptions per-scene are caught and skipped to prevent one bad scene from breaking the whole response.

routers/analytics.py — _flesch()

Pure function computing Flesch Reading Ease and Flesch-Kincaid Grade Level for a block of text. Uses _count_syllables() on each word and _sentence_stats() for sentence length. Returns (0.0, 0.0) for empty or punctuation-only input. Ease is clamped to [0, 100].

routers/sync.py — _do_pg_dump()

Generates a portable SQL backup by iterating all ORM tables and rows, serialising values via _pg_literal(). Deliberately omits openrouter_api_key and ai_providers_cfg columns so encrypted secrets never leave the machine. Output is a plain .sql file readable by restore_from_dump().

routers/sync.py — restore_from_dump()

Reads the SQL backup, saves the machine-local encrypted keys before restore, replays all SQL statements via _iter_sql_statements() (which correctly handles semicolons inside E'' strings), then re-applies the saved keys. Ensures API keys survive cross-machine restores.

routers/sync.py — _iter_sql_statements()

Generator that splits a SQL dump into individual statements. Correctly handles: semicolons inside E'' escape strings, doubled single-quotes (E'it''s ok'), backslash escapes (\n, \t), and comment lines (-- ...). Trailing statements without a semicolon are yielded.

routers/sync.py — _pg_literal()

Converts a Python value to a PostgreSQL literal string for the SQL dump. None → NULL, bool → TRUE/FALSE, int/float → numeric repr, str → E'...' with backslash and quote escaping.

routers/ai.py — generate()

POST /api/ai/generate. Streams text from the active AI provider (OpenRouter or local). Builds a context window from the current scene, adjacent scenes, and selected codex entries. Supports modes: continue, rewrite, brainstorm, and ask. Returns a Server-Sent Events stream.

routers/research.py — patch_research()

PATCH /api/research/{id}. Uses model.model_fields_set to distinguish 'field explicitly set to None' from 'field omitted from request body', so that sending linked_scene_id=null actually unlinks the scene rather than being ignored.

routers/export.py — export_project()

POST /api/export/{id}/export. Renders the manuscript as Markdown, then calls the Pandoc Docker sidecar with the chosen ExportOptions (font, size, line-spacing, page numbers, drop caps, etc.). Supports PDF, EPUB, DOCX, Markdown, and LaTeX output.

routers/scene_commands.py — sync_commands()

POST /api/scene_commands/scenes/{id}/commands/sync. Accepts a list of currency/item commands (add, remove, set) for a scene, persists them as SceneCommand rows, and returns the updated balances and logs for the requesting character.

Appendix B — Built-in Publisher Profiles

Foliantica ships with **38 publisher / agent formatting profiles** seeded at startup. These are read-only reference profiles (is_active = True). Users can create additional custom ExportProfiles per project.

Standard Manuscript

SMF Times New Roman, SMF Courier

US Trade Publishers

Berkley / PRH, Harlequin

UK Publishers

Pan Macmillan, Hachette UK, Bloomsbury, Tor UK, Soho Press

Literary Agencies

Curtis Brown, Janklow & Nesbit, Writers House

Self-Publishing

Amazon KDP, Draft2Digital

German Publishers (10)

Rowohlt, S. Fischer, Suhrkamp, Hanser, Piper, Droemer Knaur, Bastei Lübbe, Aufbau, Ullstein, dtv

French Publishers (8)

Gallimard, Seuil, Flammarion, Albin Michel, Actes Sud, Bragelonne, L'Atalante

Spanish Publishers (7)

Planeta, Alfaguara, Anagrama, Tusquets, Siruela, Ediciones Urano, Roca Editorial

Appendix C — Pydantic Schema Reference

ProjectOut

Fields: id, title, description, book_meta (BookMeta), cover_image, main_plot_color, shared_codex_project_id, created_at, updated_at

BookMeta

Fields: author, author_sort, subtitle, language (BCP 47), publisher, published_date, isbn, rights, series, series_index, genre, synopsis, translator

SceneOut

Fields: id, chapter_id, title, content, synopsis, word_count, scene_type, card_color, subplot, global_order, pov_character_id, scene_time (JSON), node_x, node_y, created_at, updated_at

CodexEntryOut

Fields: id, project_id, name, aliases, entry_type, description, notes, color, tags, is_main_char, inventory, image_path, share_mode, share_future, created_at, updated_at

ExportOptions

Fields: format (pdf|epub|docx|markdown|latex), font, font_size, line_spacing, text_align, heading_align, paragraph_indent, page_numbers, include_act_headings, include_chapter_headings, include_scene_titles, drop_caps

AIGenerateRequest

Fields: scene_id, mode (continue|rewrite|brainstorm|ask), prompt (str), context_scenes (list[int]), codex_entries (list[int])

SettingsOut

Fields: openrouter_api_key (masked), default_model, theme, language, all feature flags, service URLs (pandoc_url, lt_url, spacy_url, vale_url), sync_mirror_enabled, sync_mirror_dir

QuerySubmission

Fields: id, project_id, agent_name, agency, email, submission_type, date_sent, response_deadline, status, notes